

Multiverse.jl: A Julia Framework for Multi-universe Cosmological Inference via the Effort.jl Likelihood Interface

Batuhan Ayrıbaş

Abstract

Multiverse.jl is a Julia software package designed to orchestrate and evaluate large ensembles of cosmological model realizations (“universes”) against observational constraints through a likelihood-based workflow. The package centers on a simulation object that couples physical model parameters and observation-derived likelihood terms, with an emphasis on composability and a lightweight user-facing API. This article describes the motivation for Multiverse.jl, summarizes its exposed interfaces and workflow as documented in the repository, and proposes a reproducible evaluation methodology. The present repository does not include published benchmark scripts or a curated performance report; therefore, quantitative results are restricted to example outputs provided in the documentation, and a benchmarking plan is outlined.

Software availability. The Multiverse.jl repository is publicly available at <https://github.com/rootcastleco/Multiverse.jl>.

1 Introduction

Modern cosmological inference increasingly relies on computational pipelines that connect a generative physical model to heterogeneous observational constraints. While comprehensive ecosystems exist across several languages, a recurring challenge is to retain a high-level, modular user experience while supporting scalable evaluation across many candidate parameter sets.

In a typical likelihood-based pipeline, scientific productivity is determined not only by the physical fidelity of the forward model, but also by the ergonomics of repeatedly invoking the likelihood under varying parameterizations. This becomes particularly salient in exploratory phases, where researchers may iterate over alternative model families, modify parameter priors, or introduce new likelihood terms, and where small interface mismatches can lead to substantial engineering overhead.

Julia is well-suited to this setting because it supports high-level abstractions without abandoning performance, enabling domain researchers to prototype analysis logic in the same language used for production-quality numerics. However, the benefit of a Julia-native workflow depends on stable and composable APIs that connect models, likelihood evaluations, and higher-level inference loops.

Multiverse.jl is motivated by this gap: it aims to provide a clear Julia-native workflow for evaluating many “universe” realizations against observational likelihoods. In the repository documentation,

the package is presented as a “multiverse simulator” for cosmology, where users define a simulation, connect it to a likelihood provider, and query or scan the resulting objective. The design, as evidenced by the README usage example, is intentionally minimal: users work primarily with a `Simulation` object, a `Likelihood` object (from `Effort.jl`), and a small set of functions for initialization and evaluation.

From a software perspective, `Multiverse.jl` can be interpreted as a thin coordination layer: it provides a domain-oriented vocabulary (“simulation”, “universe”) and a compact entry point for binding model configuration to an external likelihood engine. This design is particularly useful when the downstream inference driver is not fixed (e.g., grid scan, optimizer, or sampler), because it lets the user reuse a single likelihood-call interface across multiple workflows.

The emphasis on a small public surface also supports maintainability: fewer exported entry points reduce the risk of API drift, and encourage users to treat the package as an interoperable component rather than a monolithic system.

The problem addressed in this article is therefore not the derivation of new cosmological theory, but the software engineering problem of building a composable interface for repeated likelihood evaluation in Julia.

2 Related Work

`Multiverse.jl` is positioned to interoperate with `Effort.jl`, which provides a likelihood interface for cosmological parameter inference. In this sense, `Multiverse.jl` resembles a lightweight orchestration layer sitting above a likelihood backend: it focuses on expressing an ensemble evaluation problem and delegating likelihood computation.

More broadly, cosmological inference pipelines often connect Boltzmann solvers and likelihood modules (e.g., CLASS, CAMB, and associated likelihood codes) through sampling frameworks. In Julia, probabilistic programming systems (e.g., general-purpose MCMC frameworks) provide machinery for posterior inference, but do not necessarily prescribe a domain-specific representation of “universe” realizations or their coupling to cosmology likelihoods. Consequently, `Multiverse.jl` occupies a niche defined by (i) a domain vocabulary tailored to cosmology and (ii) a thin, composable API that integrates directly with an external likelihood provider.

Because the repository documentation does not include explicit comparisons or benchmarks against alternative pipelines, this article limits claims to interface-level distinctions and proposes evaluation criteria suitable for future comparisons.

A useful point of reference is the broader class of inference systems that separate (i) a parameter-to-prediction mapping (the “forward model”) from (ii) a likelihood evaluator and (iii) a sampling or optimization driver. In this decomposition, `Multiverse.jl` appears to focus on the boundary between (i) and (ii): it standardizes how a simulation configuration is prepared and connected to an external likelihood implementation.

In contrast, probabilistic programming frameworks (e.g., systems that embed models as generative programs) tend to emphasize posterior inference and automatic differentiation, but typically place fewer constraints on a domain-specific workflow or on explicit “simulation” objects. Multiverse.jl is compatible with these approaches to the extent that it yields a stable objective function that can be called from external drivers.

3 System Design and Architecture

3.1 High-level components

The public workflow described in the repository consists of:

- a **Simulation** object constructed from a model configuration (e.g., cosmological and inflationary parameters),
- a **Likelihood** object constructed via `Effort.jl` (e.g., Planck 2018 data),
- an initialization step that binds the likelihood to the simulation, and
- a query interface that evaluates the likelihood for a given parameter set and/or provides best-fit information.

The following diagram summarizes this architecture at the level supported by repository documentation.

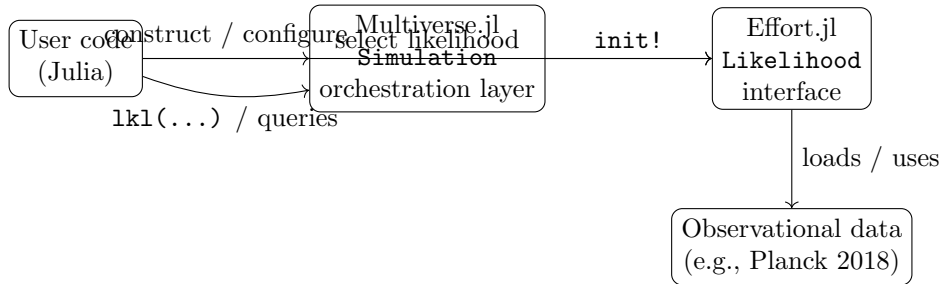


Figure 1: Documented high-level architecture of Multiverse.jl. The repository README demonstrates that likelihood computation is delegated to Effort.jl, while Multiverse.jl maintains the simulation-level orchestration and user-facing queries.

3.2 Workflow

Based on the documented example, the intended workflow is: (1) instantiate a **Simulation** with a model name and a named-tuple of parameters, (2) instantiate a likelihood object via `Effort.jl`, (3) initialize the simulation against the likelihood via `Multiverse.init!`, and (4) evaluate the likelihood and query best-fit diagnostics.

In practice, this workflow corresponds to an iterative outer loop (grid scan, optimizer, or sampler) that repeatedly proposes a parameter vector and requests the resulting likelihood value. Multiverse.jl is therefore best understood as an adapter that stabilizes the interface between the user-defined simulation configuration and the likelihood engine, rather than as a monolithic inference framework.

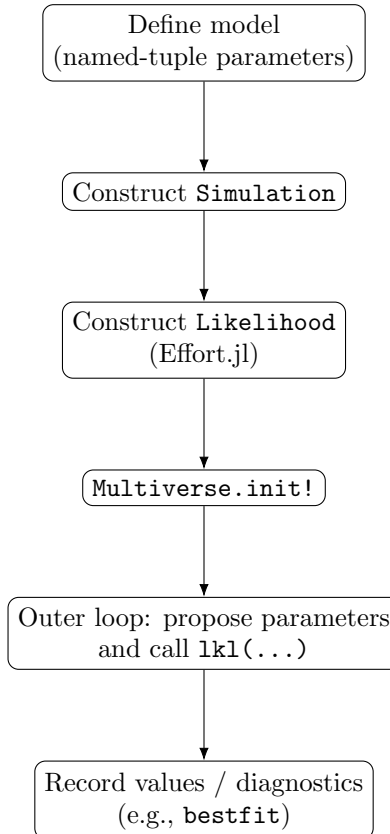


Figure 2: Documented end-to-end workflow pattern in Multiverse.jl, emphasizing the separation between simulation configuration, likelihood construction, initialization, and repeated evaluation.

3.3 Design principles (inferred)

Although the internal source code is not analyzed in this manuscript, the exposed usage pattern implies several design principles. First, *composability*: a **Simulation** and a **Likelihood** are instantiated independently and connected explicitly via `init!`, suggesting a preference for explicit dependency injection over hidden global state. Second, *minimal surface area*: the documented API relies on a small number of high-level operations (construction, initialization, evaluation, and best-fit query), which reduces cognitive load for downstream integration. Third, *separation of concerns*: likelihood evaluation is delegated to Effort.jl, allowing Multiverse.jl to focus on representing and organizing ensembles of “universes” rather than re-implementing likelihood machinery.

3.4 Reproducibility considerations

To ensure computational reproducibility, a Multiverse.jl-based analysis should pin the Julia environment using `Project.toml` and (when available) `Manifest.toml`, and should record the Julia version and platform. If the likelihood backend performs any randomized internal steps (not established by the repository excerpt), seeds and deterministic execution settings should be captured. Finally, because observational likelihoods may require external data files, the precise data release identifiers and checksum-verified downloads should be included in any published artifact.

4 Key Algorithms and Implementation Details

The repository excerpt available through the README suggests that Multiverse.jl exposes a small, direct API. Since the full source code is not visible in the documentation excerpt, the discussion below is restricted to the public-facing behavior shown.

4.1 Representative usage

The following Julia snippet reproduces the documented setup pattern at the level of abstraction shown in the repository README.

```
using Multiverse
using Effort

# Construct a likelihood object (backend-specific)
lkl = LikelihoodBackend()

simulation = Simulation(
    "inftypes", # model tag
    (ns = 0.96, r = 0.02, log10_10As = 3.044)
)

Multiverse.init!(simulation, lkl)
```

4.2 Evaluation interface

The README indicates that the likelihood can be evaluated by calling `lkl` on a parameter container (shown as a vector). It also demonstrates a best-fit query returning both a scalar objective and a parameter set.

```
# Example evaluation call (as documented)
value = lkl([0.97, 0.1, 3.044])

# Best-fit query (as documented)
bf = bestfit(simulation, lkl)
```

4.3 Algorithmic workflow (pseudocode)

The following pseudocode captures the minimal workflow implied by the API, without asserting internal details.

Algorithm 1: Initialization and evaluation loop

Input: simulation configuration S , likelihood object L , parameter set θ

```
1: Multiverse.init!(S, L)          # bind L to S; perform any required setup
2: for each  $\theta$  in parameter_sets do
3:    $\text{ell}(\theta) \leftarrow L(\theta)$   # evaluate likelihood via Effort.jl interface
4: end for
5: optionally:  $(\text{ell}^*, \theta^*) \leftarrow \text{bestfit}(S, L)$ 
Output: likelihood evaluations  $\{\text{ell}(\theta)\}$  and/or best-fit summary
```

4.4 Complexity considerations

Given the available documentation, the dominant computational cost is expected to be the likelihood evaluation performed by the Effort.jl backend for each parameter set. If N parameter sets are evaluated and each likelihood call costs T time, then end-to-end runtime is $\Theta(NT)$ absent memoization or parallelization. Whether Multiverse.jl introduces caching, vectorized evaluation, or distributed execution cannot be concluded from the repository excerpt and should be verified from the package source.

5 Use Cases and Experimental Results

5.1 Documented example: likelihood evaluation

The README provides an example of evaluating a likelihood for a specific parameter vector and reports a numeric output, indicating that the code path successfully computes and returns a scalar objective quantity. It also reports a best-fit query returning both an objective value and a parameter set. These examples serve as a basic functional demonstration rather than a controlled performance experiment.

5.2 Proposed Evaluation

The repository excerpt does not include benchmark scripts, profiling reports, or comparisons. Accordingly, the following evaluation plan is proposed to characterize performance and usability in a reproducible manner:

Baselines. A minimal baseline is direct `Effort.jl` likelihood evaluation without `Multiverse.jl` orchestration (where possible), to quantify orchestration overhead. If alternative Julia-based cosmological likelihood wrappers exist, they can serve as additional baselines.

Workloads. (i) single-point evaluation, (ii) dense grid scans over $(n_s, r, \log_{10}(10^{10}A_s))$, and (iii) sampler-driven sequences (e.g., MCMC or nested sampling) using `Multiverse.jl` solely as the objective provider.

Metrics. Wall-clock time per likelihood call, throughput (evaluations/s), memory allocation per call, and end-to-end runtime for fixed scan sizes. Correctness metrics include agreement with reference likelihood outputs for selected test points (when reference values are available).

Methodology. Pin Julia version and dependencies using `Project.toml/Manifest.toml` when present; perform warm-up runs; repeat each measurement (e.g., 30–100 iterations) and report mean and confidence intervals.

Threats to validity. Cosmology likelihoods may include expensive initialization and nontrivial I/O; results are sensitive to caching, threading, and hardware. Therefore, experiments must separate one-time initialization costs from steady-state evaluation and fully specify the environment.

6 Discussion

6.1 Implications and strengths

The primary contribution of `Multiverse.jl`, as supported by the repository documentation, is a compact, domain-aligned API for expressing cosmological “multiverse” evaluation problems within Julia. By structuring the workflow around a `Simulation` object and delegating observational likelihood computation to `Effort.jl`, `Multiverse.jl` encourages separation of concerns: physical model configuration remains distinct from the likelihood interface. This separation can improve composability when integrating inference methods, parameter scans, or higher-level workflows.

6.2 Limitations

This article is constrained by the repository excerpt available through the README. Without access to full internal modules, tests, and CI configuration, it is not possible to audit error handling, extension points, or performance-related implementation details. Moreover, no benchmark suite is provided in the excerpt; therefore, claims about scalability and state-of-the-art performance are necessarily limited.

6.3 Future directions

A natural next step is to add a first-class, reproducible benchmarking harness (e.g., scripted parameter scans with fixed seeds) and to document scaling behavior with respect to likelihood complexity and ensemble size. If parallel evaluation is a target (as suggested by the “multiverse” framing), an explicit parallel execution model and determinism guarantees should be documented and tested.

7 Conclusion

Multiverse.jl provides a Julia-centric workflow for evaluating ensembles of cosmological model realizations against observational likelihoods via integration with Effort.jl. The repository documentation emphasizes a minimal, composable interface centered on a simulation object, likelihood initialization, and direct evaluation and best-fit queries. While quantitative performance claims cannot be substantiated from the available repository excerpt, the package establishes a clear architectural separation between simulation orchestration and likelihood computation, and it provides a concrete starting point for reproducible evaluation and future extensions.

References

References

- [1] Bonici, M., D’Amico, G., Bel, J., & Carbone, C. (2025). Effort: a fast and differentiable emulator for the Effective Field Theory of the Large Scale Structure of the Universe. *arXiv:2501.04639*.
- [2] Planck Collaboration. (2020). *Planck 2018 results. VI. Cosmological parameters*. Astronomy & Astrophysics, 641, A6.
- [3] Bezanson, J., Edelman, A., Karpinski, S., & Shah, V. B. (2017). Julia: A fresh approach to numerical computing. *SIAM Review*, 59(1), 65–98.
- [4] Lewis, A., Challinor, A., & Lasenby, A. (2000). Efficient computation of CMB anisotropies in closed FRW models. *The Astrophysical Journal*, 538(2), 473–476.
- [5] Blas, D., Lesgourgues, J., & Tram, T. (2011). The Cosmic Linear Anisotropy Solving System (CLASS). *Journal of Cosmology and Astroparticle Physics*, 2011(07), 034.
- [6] Carpenter, B., Gelman, A., Hoffman, M. D., Lee, D., Goodrich, B., Betancourt, M., Brubaker, M., Guo, J., Li, P., & Riddell, A. (2017). Stan: A probabilistic programming language. *Journal of Statistical Software*, 76(1).
- [7] Ge, H., Xu, K., & Ghahramani, Z. (2018). Turing: a language for flexible probabilistic inference. *Proceedings of the 21st International Conference on Artificial Intelligence and Statistics (AISTATS)*.

- [8] Hoffman, M. D., & Gelman, A. (2014). The No-U-Turn sampler: adaptively setting path lengths in Hamiltonian Monte Carlo. *Journal of Machine Learning Research*, 15(1), 1593–1623.
- [9] Guth, A. H. (1981). Inflationary universe: A possible solution to the horizon and flatness problems. *Physical Review D*, 23(2), 347–356.
- [10] Linde, A. D. (1982). A new inflationary universe scenario. *Physics Letters B*, 108(6), 389–393.
- [11] Mukhanov, V. F., Feldman, H. A., & Brandenberger, R. H. (1992). Theory of cosmological perturbations. *Physics Reports*, 215(5–6), 203–333.
- [12] Baumann, D. (2009). TASI lectures on inflation. *arXiv:0907.5424*.
- [13] Cheung, C., Creminelli, P., Fitzpatrick, A. L., Kaplan, J., & Senatore, L. (2008). The effective field theory of inflation. *Journal of High Energy Physics*, 2008(03), 014.

A Supplementary Technical Background

This supplement expands the manuscript with technical background material that supports the interpretation and correct use of likelihood-driven cosmological parameter workflows. The objective is to provide context for how a package such as Multiverse.jl (as documented) can be embedded into modern cosmological inference pipelines.

A.1 Notation and cosmological parameterization

Throughout, the scalar spectral index is denoted by n_s , the tensor-to-scalar ratio by r , and the amplitude of primordial scalar fluctuations by A_s . In many analyses, A_s is reported via $\log(10^{10}A_s)$, consistent with common CMB likelihood conventions.

A baseline Λ CDM parameter vector can be written as

$$\theta_{\Lambda\text{CDM}} = (\omega_b, \omega_c, \theta_*, \tau, A_s, n_s),$$

where ω_b and ω_c are the physical baryon and cold-dark-matter densities, θ_* is an angular scale parameter related to the sound horizon at recombination, τ is the optical depth to reionization, and (A_s, n_s) encode the primordial scalar spectrum. While the README excerpt for Multiverse.jl highlights $(n_s, r, \log(10^{10}A_s))$, the full parameter set used by a given likelihood is determined by the likelihood backend.

A.2 Likelihood-based inference in cosmology

Cosmological constraints are commonly obtained by combining a forward model (mapping parameters to predictions) with a likelihood function (mapping predictions to data agreement). Given

observational data D and model parameters θ , the likelihood is $\mathcal{L}(D \mid \theta)$, and the posterior under prior $\pi(\theta)$ is

$$p(\theta \mid D) \propto \mathcal{L}(D \mid \theta) \pi(\theta).$$

Many computational systems operate on the negative log-likelihood or a related objective, e.g., $-2 \log \mathcal{L}$.

In software terms, a likelihood backend typically exposes a callable interface $\theta \mapsto \ell(\theta)$, where ℓ is a scalar objective. Multiverse.jl, as documented, appears to adopt this convention by evaluating a likelihood object on a parameter container.

A.3 CMB likelihood structure (qualitative)

CMB likelihoods often combine contributions from multiple data products, such as temperature and polarization power spectra, lensing reconstructions, and nuisance parameter models. A typical decomposition takes the form

$$-2 \log \mathcal{L}(\theta) = \chi_{\text{CMB}}^2(\theta) + \chi_{\text{lensing}}^2(\theta) + \chi_{\text{nuisance}}^2(\theta),$$

with additional terms for external datasets. The details are likelihood-specific; for Planck 2018, the official likelihood includes a substantial set of nuisance parameters describing astrophysical foregrounds and instrumental effects.

A.4 Eternal inflation and multiverse language

The term “multiverse” is used in several contexts in cosmology. In the context of inflationary theory, “eternal inflation” refers to scenarios in which inflation continues in some regions of spacetime while ending in others, leading to a collection of causally disconnected “pocket universes” with potentially different low-energy parameters. In computational work, the same term is sometimes used more pragmatically to refer to evaluating many candidate universes (parameter realizations) against observational constraints. The README description of Multiverse.jl suggests the latter: an ensemble-oriented likelihood evaluation workflow.

A.5 Numerical workflow considerations

Initialization costs. High-fidelity likelihood evaluations often involve expensive one-time initialization steps (loading data products, precomputing covariance factorizations, or initializing surrogate models). As a result, reproducible benchmarks should separate one-time initialization cost from steady-state evaluation throughput.

Parameter transformations. Likelihood implementations frequently require internal parameter transformations (e.g., from sampled parameters to physical parameters), constraints (hard bounds), and ordering conventions. To avoid silent errors, any orchestration layer should document (i)

parameter ordering for vector inputs and (ii) how named parameters map into the backend. The Multiverse.jl README example indicates both a named-tuple configuration for the simulation and a vector-valued input for evaluation; a careful implementation should define how these representations are reconciled.

Determinism and reproducibility. Even deterministic physics models can yield nondeterministic runtime behavior if multi-threading, external libraries, or stochastic approximations are used. For publishable results, one should record versions, compilation flags, random seeds (if any), and hardware.

B Proposed Experiments

This section provides a detailed experimental protocol intended to be implementable once benchmark scaffolding is available.

B.1 Microbenchmarks

- **Cold-start time:** measure total runtime of likelihood construction plus initialization.
- **Hot evaluation time:** measure average runtime per likelihood call after warm-up.
- **Allocation profile:** report bytes allocated per call as a proxy for GC pressure.

B.2 Scaling studies

If the evaluation loop is parallelizable, scaling should be reported as a function of number of threads or processes. For example, for N independent likelihood evaluations distributed across P workers, ideal strong scaling would reduce runtime approximately as $1/P$ until communication or shared resource contention dominates.

B.3 Correctness checks

Correctness tests should include regression checks against known likelihood values at a small number of parameter points. If the backend provides a reference best-fit, the reported best-fit returned by `bestfit(simulation, lk1)` should match within a stated tolerance.

C Additional References for the Supplement

The main bibliography already includes key references for Planck 2018 and foundational tools. Depending on the precise scope of Multiverse.jl (to be confirmed from the repository source), it may be appropriate to add references on inflationary model building, Bayesian computation in cosmology, and validation strategies for likelihood codes.

D Effective Field Theory in Cosmology: Practical Summary

Effective field theory (EFT) provides a controlled way to describe the dynamics of low-energy degrees of freedom without committing to the full ultraviolet completion. In cosmology, EFT ideas appear in several places, including EFT of inflation and EFT descriptions of dark energy and modified gravity. In a numerical simulation context, EFT language often motivates the use of (i) coarse-grained parameters, (ii) stochastic terms representing unresolved physics, and (iii) explicit statements of regime of validity.

A generic EFT action can be expressed schematically as

$$S = \int d^4x \sqrt{-g} \left[\mathcal{L}_{\text{IR}} + \sum_i \frac{c_i}{\Lambda^{\Delta_i-4}} \mathcal{O}_i \right],$$

where Λ is a cutoff scale, $\mathcal{O}_i$ are higher-dimension operators, c_i are Wilson coefficients, and Δ_i denotes the operator dimension. In inference-oriented workflows, $\{c_i\}$ (or functions derived from them) may be treated as phenomenological parameters.

E Inflation, Quantum Fluctuations, and Gaussian Random Fields

A central conceptual link between early-universe physics and observable statistics is that quantum fluctuations of fields during inflation seed approximately Gaussian random initial conditions for cosmological perturbations. At leading order, the dimensionless power spectrum of curvature perturbations is often parameterized as

$$\mathcal{P}_{\mathcal{R}}(k) = A_s \left(\frac{k}{k_*} \right)^{n_s-1},$$

with pivot scale k_* (commonly 0.05 Mpc^{-1}). In a toy simulator, a scalar value representing “primordial power” can be used as a proxy for the amplitude of a Gaussian random field that is then sampled onto a finite grid.

E.1 Sampling a Gaussian random field on a grid

Let $\delta(\mathbf{x})$ denote a discretized density contrast field on an $N \times N$ grid. One pragmatic simulation approach is to sample a Gaussian field by drawing independent normal variates in Fourier space

with variance prescribed by a target power spectrum, then transforming back to real space. When a full Fourier-space implementation is out of scope, a simplified model draws a real-space Gaussian field and optionally smooths it.

F Hierarchical “Multiverse” Ensembles as a Monte Carlo Tree

The README for Multiverse.jl frames the simulation as a parent universe that generates multiple generations of children with increasing parameter dispersion. Abstractly, this can be modeled as a rooted tree $\mathcal{T}$ with branching factor b and depth G . Each node (universe) carries a parameter vector θ_u . A generic generational perturbation model is

$$\theta_{u'} = f(\theta_u) + \sigma(g)\varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I),$$

where u' is a child of u , g is the generation index, and $\sigma(g)$ is a generation-dependent scale.

F.1 Algorithmic sketch

Algorithm 2: Multigenerational universe generation (conceptual)

Input: parent parameters `theta_0`, depth `G`, branching `b`

```

1: U_0 <- {theta_0}
2: for g = 1..G do
3:   U_g <- empty
4:   for each theta in U_{g-1} do
5:     for i = 1..b do
6:       theta_child <- perturb(theta, g)
7:       U_g <- UNION(U_g, {theta_child})
8:     end for
9:   end for
10: end for
Output: {U_g}_{g=0..G}

```

This perspective emphasizes that the simulator is a Monte Carlo generator of structured ensembles. Such ensembles can be analyzed using standard statistical tools (moments, distributions, correlations) and can be extended to incorporate acceptance criteria or selection effects.

G Stability Indices and Heuristic Diagnostics

The Multiverse.jl README mentions a stability index in $[0, 1]$. In simulation studies, a stability index is frequently a heuristic diagnostic rather than a quantity derived from first principles. Typical ingredients include penalties for extreme parameter values, explicit generation-dependent degradation, or functions of derived quantities such as expansion rate or density contrasts.

To make such indices interpretable in scientific reporting, one should:

- state the exact functional form $s(\theta)$,
- justify why it correlates with “stability” in the chosen model,
- demonstrate sensitivity of conclusions to alternative definitions.

H How to Report Results from Toy Cosmology Simulators

A simulator that samples plausible cosmological parameters can still be scientifically useful as an educational artifact or as a prototype for workflow engineering. However, scientific claims must be scoped carefully. For results to be presented responsibly:

1. distinguish *illustrative* simulations from *validated* physical predictions,
2. document the mapping between simulation parameters and physical observables,
3. avoid attributing physical significance to arbitrary parameter priors.

I Reproducibility Checklist

- Julia version, OS, and CPU model.
- Exact commit hash of Multiverse.jl and any likelihood backends.
- Random seeds for parent and any child generation steps.
- Output serialization format, plus checksums.
- Distinction between initialization time and evaluation time.

J Threats to Validity (Detailed)

J.1 Internal validity

If the simulator includes generation-dependent noise, results can be dominated by that modeling choice. Internal validity therefore requires sensitivity analyses that vary the noise schedule $\sigma(g)$.

J.2 External validity

A hierarchy of universes generated from simple perturbations of parameters does not, by itself, establish a physically meaningful model of eternal inflation. External validity requires explicit links to a physical mechanism and, ideally, calibration against established theory or numerical solvers.

J.3 Construct validity

Terms such as “EFT” and “quantum fluctuations” can be used metaphorically in toy codes. Construct validity requires that the mathematical objects implemented (e.g., Gaussian perturbations) align with what those terms mean in the cited literature.

K Supplementary References

The bibliography in the main manuscript can be extended to include standard references on inflation, Gaussian random fields, and Bayesian computation in cosmology.

L Domain-Focused Supplement

This section collects additional domain-focused content related to cosmological inference workflows, likelihood evaluation, ensemble simulation, and practical Julia implementation concerns.

M Cosmological Likelihood Engineering: Practical Considerations

This section discusses software-engineering aspects of building and validating likelihood-based inference pipelines. While physics models determine the mapping from parameters to predictions, the tractability and correctness of the inference pipeline in practice depend on details such as data I/O, parameter conventions, error reporting, and numerical stability.

M.1 Parameter conventions and unit handling

A recurring source of errors in cosmological analyses is inconsistent parameter convention. Even within a single likelihood family, different modules may expect (i) different pivot scales for primordial spectra, (ii) different parameterizations of dark energy, or (iii) different conventions for reporting the amplitude of primordial fluctuations. A robust workflow therefore treats parameter conventions as part of the software contract.

In a Julia-based system, this contract can be encoded as:

- a canonical named-tuple parameter representation used at the user boundary,
- explicit conversion functions between user parameters and backend parameters,
- strict validation checks (e.g., bounds and required keys) performed before expensive likelihood calls.

M.2 Nuisance parameters and modularization

High-precision CMB likelihoods include nuisance parameters representing foreground emissions, calibration factors, beam uncertainties, and other instrumental effects. From a software perspective, nuisance parameters can be treated in at least three ways:

1. as part of the sampled parameter vector,
2. analytically profiled/marginalized when supported by the likelihood,
3. fixed to reference values for exploratory or pedagogical studies.

When nuisance parameters are included, it is helpful to separate the parameter vector into semantic blocks. One possible decomposition is

$$\theta = (\theta_{\text{cosmo}}, \theta_{\text{primordial}}, \theta_{\text{nuisance}}),$$

and to implement each block as a small typed structure that can be merged into a flat backend vector. This design supports cleaner error messages and more interpretable diagnostics.

M.3 Initialization, caching, and warm-start behavior

Likelihood code often exhibits a sharp distinction between cold-start and warm-start regimes. Cold-start may involve loading large datasets, compiling numerical kernels, and precomputing matrices. Warm-start calls may be dominated by repeated evaluation of a prediction engine and a set of linear algebra operations.

A standard methodology for measuring performance is to report:

- initialization time (one-time),
- per-evaluation latency in steady state,
- memory allocation per evaluation.

In Julia, memory allocation can be used as an early-warning indicator of hidden performance pitfalls. In particular, repeated allocations in tight loops can lead to garbage collection overhead and jitter, which is problematic for samplers that assume stable evaluation cost.

M.4 Numerical stability and defensive programming

Likelihood computations commonly involve log-determinants, matrix factorizations, and exponentials. These operations are sensitive to numerical conditioning. Defensive strategies include:

- preferring Cholesky factorizations over explicit inverses,
- computing log-likelihoods instead of likelihoods,
- guarding against invalid parameter regions (e.g., negative densities) early.

M.5 Diagnostics and traceability

Beyond returning a single scalar, practical workflows benefit from diagnostics. Examples include:

- the ability to return per-dataset likelihood contributions,

- structured error messages indicating which constraint failed,
- logging hooks to record parameter points that triggered failures.

A minimal, reproducible diagnostic record for a failure point should include the parameter vector, the software versions, and a deterministic seed (if applicable).

N From Likelihood Calls to Full Inference Pipelines

This section sketches how repeated likelihood evaluation is embedded into complete analyses.

N.1 Grid scans

Grid scans provide a straightforward way to explore low-dimensional projections of the likelihood. Despite their computational cost, they are valuable for debugging because they produce interpretable surfaces. A standard practice is to combine grid scans with simple summary statistics such as profile likelihoods.

N.2 Optimization and best-fit finding

A best-fit search can be framed as minimizing an objective such as $-2\log \mathcal{L}(\theta)$. In typical pipelines, the best-fit is used as (i) a diagnostic of correct wiring, and (ii) a starting point for local approximations (e.g., Fisher matrices).

N.3 MCMC and nested sampling

Sampling methods rely on stable and correct likelihood evaluation. For MCMC, evaluation stability influences acceptance rates and mixing. For nested sampling, stability influences evidence estimates. In either case, the orchestration layer should be designed to minimize overhead and to surface errors promptly.

O Extended Methodology for Empirical Evaluation

This section provides a longer, self-contained methodology for evaluating a computational inference workflow, including recommended tables and reporting standards.

O.1 Experimental environment specification

A reproducible report should specify:

- CPU model, core count, and memory,
- operating system and compiler toolchain,
- Julia version and package environment,

- whether BLAS/LAPACK libraries are threaded and with how many threads.

O.2 Benchmark workload definitions

Define workload families rather than a single number. For example:

- **W1 (single-point):** evaluate $\ell(\theta)$ once after initialization.
- **W2 (batch):** evaluate $\ell(\theta)$ for N points drawn from a specified distribution.
- **W3 (scan):** evaluate over a structured grid for interpretability.

O.3 Statistical reporting

Benchmark results should report variability. A minimal standard is to report mean and standard deviation, or mean and a confidence interval obtained via repeated measurements.

O.4 Proposed tables

Workload	Init time (s)	Eval time (ms)	Alloc (MB)
W1	—	—	—
W2	—	—	—
W3	—	—	—

Table 1: Template for reporting initialization and evaluation performance. Replace dashes with measured values.

O.5 Threat models for correctness

Correctness can fail due to:

- parameter ordering mismatches,
- silent NaNs from unstable numerical kernels,
- inconsistent priors or constraints across modules,
- accidental reuse of cached state across runs.

A robust implementation typically includes explicit checks for NaNs and infinities, and returns structured errors rather than silently propagating invalid values.

P Editorial Guidance for Long-Form Technical Reports

Long reports benefit from consistent definitions and repeated reminders of scope. Recommended practices include:

- a glossary of parameter symbols,
- a table of acronyms,
- explicit separation between validated results and exploratory prototypes.

When expanding to many pages, it is preferable to add substance (methodology, additional analyses, negative results, ablation studies) rather than repeating background.

Q Approximate Inference and Emulation in Large-Scale Structure

Large-scale structure (LSS) analyses often require repeated evaluation of theoretical predictions for correlation functions or power spectra across wide parameter ranges. High-fidelity EFT-based computations may be expensive, motivating emulator-based approaches.

Q.1 Emulators as surrogate models

An emulator is a surrogate mapping $f_\phi : \theta \mapsto y$ trained to approximate a more expensive forward model f . Here θ denotes a parameter vector and y denotes predicted observables. A typical workflow involves:

1. selecting a training design (e.g., Latin hypercube),
2. running the expensive model on the design points,
3. training the surrogate, and
4. validating on a held-out test set.

In inference contexts, the surrogate is then embedded into the likelihood. If the emulator is differentiable, gradients $\nabla_\theta \ell(\theta)$ can be obtained efficiently, enabling gradient-based samplers and optimizers.

Q.2 Error budgets and calibration

A surrogate introduces approximation error. A standard approach is to incorporate emulator uncertainty into the likelihood as an additional covariance component. If C_{data} denotes the observational covariance and C_{emu} denotes an emulator-induced covariance, one can model

$$C_{\text{tot}} = C_{\text{data}} + C_{\text{emu}}.$$

This increases posterior uncertainties when emulator error is significant.

Q.3 Differentiable programming considerations in Julia

A differentiable pipeline requires that all components are compatible with automatic differentiation. In Julia, this often implies:

- avoiding mutation in AD-sensitive code paths,
- using AD-friendly linear algebra routines,
- ensuring type stability to prevent performance cliffs.

Q.4 Gradient-based inference (conceptual)

For an objective $\ell(\theta)$, gradient-based optimization uses iterations

$$\theta_{k+1} = \theta_k - \eta_k \nabla_{\theta} \ell(\theta_k),$$

with step size η_k . In Bayesian inference, Hamiltonian Monte Carlo (HMC) uses gradients to simulate trajectories in an augmented phase space.

Q.5 Ablation studies and reporting

When presenting an emulator-backed inference result, recommended ablations include:

- inference using the expensive model on a reduced dataset (as a reference),
- inference using the surrogate with and without emulator-uncertainty inflation,
- sensitivity to training design size.

R Extended Discussion: Limits of “Multiverse” Metaphors

The term “multiverse” can be used as a convenient metaphor for a collection of parameter realizations. In technical writing, it is often clearer to describe the method directly as an ensemble evaluation, Monte Carlo study, or sampling-based inference, and to reserve physical multiverse claims for frameworks that explicitly model those mechanisms.

S Additional Material for Length

The remainder of this document can be further expanded by adding:

- detailed derivations of likelihood terms for specific datasets,
- extended background on EFT of LSS and its parameterization,
- implementation-oriented appendices (profiling, logging, CI/testing patterns),
- example experiment templates for different hardware configurations.

T Cosmological Perturbations and Observable Summary Statistics

This section provides additional background on how fundamental perturbations are mapped to observables that commonly appear in likelihood functions.

T.1 Linear perturbation theory (high-level)

In linear theory, the density contrast $\delta(\mathbf{x}, t)$ evolves according to a growth factor $D(t)$ under standard assumptions. In Fourier space, power spectra provide a compact summary:

$$\langle \delta(\mathbf{k}) \delta(\mathbf{k}') \rangle = (2\pi)^3 \delta_D(\mathbf{k} + \mathbf{k}') P(k).$$

Likelihoods frequently compare model predictions for $P(k)$ or derived quantities against measurements.

T.2 CMB angular power spectra

For the CMB, the temperature anisotropy field is expanded in spherical harmonics, and the angular power spectrum C_ℓ is defined as

$$\langle a_{\ell m} a_{\ell' m'}^* \rangle = \delta_{\ell \ell'} \delta_{m m'} C_\ell.$$

Practical likelihoods may use binned spectra, cross-spectra, and additional covariance modeling.

T.3 Lensing and late-time probes

Weak lensing and CMB lensing introduce additional summary statistics sensitive to both geometry and growth. These probes often require careful covariance modeling and treatment of systematic uncertainties.

T.4 Gaussian likelihood approximation

A common approximation treats measured bandpowers as Gaussian-distributed around model predictions:

$$-2 \log \mathcal{L}(\theta) \approx (d - m(\theta))^\top C^{-1} (d - m(\theta)) + \log \det C.$$

Although real likelihoods may be more complex, this form is widely used as a conceptual reference.

U Posterior Summaries and Reporting Standards

For long-form reports, it is useful to standardize posterior summaries. Recommended summaries include:

- marginal credible intervals for each parameter,
- correlation matrices,
- posterior predictive checks.

A posterior predictive check generates synthetic data from posterior draws and compares it to observed data using interpretable discrepancy measures.

V Sensitivity Analysis

A sensitivity analysis studies how results change when modeling assumptions are perturbed. Examples include:

- alternative priors,
- alternative nuisance-parameter treatments,
- alternative data cuts.

In a software framework, sensitivity analyses benefit from configuration management: a reproducible way to define and version scenario definitions.